STEP 3 · 2주차

주석과 모델 학습

폴리곤을 그리고 YOLOv8s-Seg를 학습시킨다

강의 50분 | 용어 정리 | 실습 6단계

이번 주 목표는 높은 성능이 아니다. 학습이 오류 없이 돌아가는 것까지가 목표다.

이번 주 진행 방식

지난주와 같은 순서로

① 강의	50분	데이터 구축과 YOLOv8s-Seg	본 강의 자료 15~27페이지
② 용어 정리	20분	포맷·전처리·학습 용어	이 자료 4~7페이지
③ 실습	70분	주석 → 변환 → 점검 → 학습	이 자료 9~15페이지
④ 과제	다음 주까지	학습을 끝까지 돌리고 가중치 저장	이 자료 18페이지

지난주 작성한 주석 기준서를 참고해 동일한 기준으로 라벨링한다.

오늘 강의에서 기억할 여섯 가지

실습 전에 다시 확인

1	**폴리곤 주석**	사각형보다 주석 비용은 크지만, 가늘고 불규칙한 균열의 실제 형상을 표현하는 데 유리하다.
2	**원본 단위 분할**	같은 원본에서 파생된 영상이 학습·검증·테스트셋에 동시에 포함되지 않도록 분할한다.
3	**증강은 학습셋만**	증강 영상은 학습셋에만 사용하고, 검증·테스트셋은 원본 영상으로 구성한다.
4	**letterbox 960**	종횡비를 유지하며 크기를 맞추고 남는 부분은 패딩으로 채운다. 가는 균열의 세부 정보 보존을 고려해 입력 크기 960을 사용한다.
5	**Mosaic·MixUp 비활성**	본 연구에서는 결함 경계와 콘크리트 표면의 공간적 맥락 왜곡을 줄이기 위해 비활성화하였다.
6	**학습 설정**	AdamW · 초기 학습률 0.00015 · batch 4 · 60 epochs

일반 객체 검출에서 자주 사용하는 증강도 결함의 형태와 연구 목적에 따라 다르게 적용할 필요가 있다.

용어 정리 ① 주석과 포맷

데이터를 모델이 읽을 형태로

폴리곤 좌표 Polygon coordinates	결함 경계를 이루는 다각형 꼭짓점의 x, y 좌표 목록 → 결함 경계를 따라 순서대로 좌표를 지정
YOLO seg 포맷 –	클래스 + 정규화된 폴리곤 좌표 → 한 줄에 하나의 인스턴스를 기록
정규화 좌표 Normalized	x좌표는 영상 너비, y좌표는 영상 높이로 나눈 0~1 값 → 해상도가 달라도 유효
data.yaml –	train/val/test 경로와 클래스 이름을 정의하는 데이터 설정 파일 → 학습 시작에 필요
라벨 파일 Label file	영상 하나당 텍스트 파일 하나 → 이미지와 같은 파일명을 사용하고 확장자는 .txt

YOLO segmentation 라벨의 polygon 좌표는 영상 크기에 대해 정규화한 0~1 범위의 상대좌표로 저장한다.

용어 정리 ② 전처리와 증강

모델에 넣기 전 처리

letterbox –	종횡비를 유지해 리사이즈한 뒤 남는 영역을 패딩으로 채우는 방식 → 형상 왜곡을 줄이면서 입력 크기를 통일
입력 크기 `imgsz`	모델 입력 영상의 기준 크기 → 본 연구에서는 960을 사용하여 가는 균열의 세부 정보 보존을 고려
Mosaic –	여러 영상을 하나의 학습 이미지로 결합하는 증강 기법 → 본 연구에서는 콘크리트 표면의 공간적 맥락 왜곡을 줄이기 위해 비활성화
MixUp –	두 영상과 라벨을 혼합해 새로운 학습 샘플을 만드는 증강 기법 → 본 연구에서는 결함 경계의 해석 가능성을 유지하기 위해 비활성화
하드 네거티브 `Hard negative`	결함이 아니지만 결함과 유사하게 보이는 비결함 사례 → 학습에 포함하면 오탐 감소에 도움을 줄 수 있음

증강의 사용 여부도 실험 설계의 일부다. 적용 또는 비활성화한 이유를 결과와 함께 설명할 수 있어야 한다.

용어 정리 ③ 학습 설정

YOLO에서 정해줘야 하는 값

에폭 Epochs	전체 학습 데이터를 몇 번 반복하여 학습할지 → 본 연구에서는 60 epochs 사용
배치 크기 Batch	한 번의 학습 단계에서 동시에 처리하는 이미지 수 → 입력 크기 960과 GPU 메모리 사용량을 고려해 4로 설정
옵티마이저 AdamW	가중치를 갱신하는 방법 → Adam에 weight decay를 결합한 AdamW 사용
학습률 Learning rate	모델 가중치를 갱신할 때 변화 크기를 결정하는 값 → 초기 학습률 lr0 = 0.00015
사전학습 가중치 Pretrained weights	사전에 학습된 가중치로 초기화하여 학습 시작 → yolov8s-seg.pt

입력 해상도가 커질수록 GPU 메모리 사용량이 증가하므로, 배치 크기와 입력 크기는 함께 조정할 필요가 있다.

용어 정리 ④ 손실 함수

학습 중 지켜볼 네 가지

Box loss –	예측 바운딩 박스와 정답 박스의 위치 차이를 나타내는 손실 → 결함 위치 예측의 정확도와 관련
Seg loss Segmentation loss	예측 마스크와 정답 마스크의 차이를 나타내는 손실 → 결함 영역과 경계 예측 성능에 관련
Cls loss Classification loss	예측 클래스와 정답 클래스의 차이를 나타내는 손실 → crack / efflorescence / spall_detach 구분 성능과 관련
DFL Distribution Focal Loss	바운딩 박스 경계 위치를 연속값이 아닌 분포 형태로 학습하는 손실 → 박스 좌표 회귀의 정밀도를 높이는 데 사용
모델 선택 Model selection	학습 중 저장된 가중치 중 최종 평가에 사용할 모델을 선택하는 과정 → 검증 성능을 기준으로 저장된 best.pt 사용

검증 손실과 성능은 학습 과정에서 변동할 수 있으므로, 학습 손실만으로 최종 모델을 결정하지 않고 검증 성능을 함께 확인한다.

오늘의 실습

여섯 단계 — 주석부터 학습까지

단계 1	**폴리곤 주석**	제공된 실습 데이터 중 일부 이미지를 CVAT 또는 Roboflow로 주석
단계 2	**포맷 확인**	YOLO segmentation 라벨 파일 형식을 직접 확인
단계 3	**데이터 무결성 점검**	영상–라벨 대응, 빈 파일, 클래스 ID, 좌표 범위를 점검
단계 4	**분할과 data.yaml**	원본 단위로 train/val/test를 분할하고 data.yaml 작성
단계 5	**학습 실행**	YOLOv8s-Seg 미세조정
단계 6	**학습 곡선 확인**	results.png에서 loss와 mask mAP 변화 확인

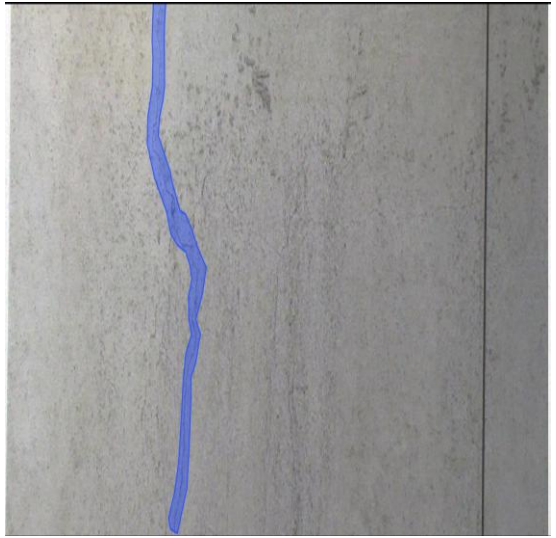
실습에서는 제공된 데이터 중 일부 이미지를 직접 주석하고, 나머지 라벨 데이터는 학습에 활용한다. 주석 수보다 기준의 일관성이 더 중요하다.

폴리곤 주석

지난주 기준서를 옆에 두고

목표	**세 클래스의 결함 경계를 폴리곤으로 그린다**
도구 선택	CVAT 또는 Roboflow 사용 (수업 환경에 맞는 도구 선택)
클래스 등록	crack · efflorescence · spall_detach 순서로 등록
주석 원칙	주석 기준서에서 제외 대상으로 정의한 요소는 라벨링하지 않는다
목표 분량	제공 데이터 중 10~20장을 직접 주석하여 라벨링 과정을 익힌다
중복 확인	동일하거나 매우 유사한 영상이 반복되지 않는지 확인
내보내기	YOLOv8 Segmentation 포맷으로 export

예시: 균열 경계를 따라 polygon 형태로 주석

	
ORIGINAL	GROUND TRUTH

포맷 확인

가장 흔한 실수를 여기서 잡는다

목표 **라벨이 세그멘테이션 포맷으로 저장되었는지 확인한다**

```
import glob

lbls = sorted(
    glob.glob("dataset/labels/**/*.txt", recursive=True)
)

print("라벨 파일 수:", len(lbls))

if not lbls:
    print("라벨 파일이 없습니다.")
else:
    line = open(lbls[0], encoding="utf-8").readline().split()

    print("토큰 개수:", len(line))
    print("앞부분:", line[:9])

    if len(line) == 5:
        print("Detection 포맷입니다.")
    elif len(line) >= 7 and (len(line) - 1) % 2 == 0:
        print("Segmentation 포맷입니다.")
    else:
        print("라벨 형식을 다시 확인하세요.")
```

확인 방법

- 한 줄이 5개 값이면 Detection 포맷인지 확인
- Segmentation 포맷은 클래스 ID 뒤에 3쌍 이상의 polygon 좌표가 존재
- 첫 번째 값은 클래스 ID 0·1·2인지 확인
- 이후 x, y 좌표가 모두 0~1 범위인지 확인
- 영상과 라벨의 대응 관계를 확인하고, 누락 여부는 다음 단계에서 점검

코드(1)

```
CLASS_NAMES = {
    0: "crack",
    1: "efflorescence",
    2: "spall_detach"
}

random.seed(SEED)

# ========================
# 원본 ID 추출 함수
# 파일명 규칙에 맞게 수정
# 예: abc_aug1.jpg -> abc
# ========================
def get_origin_id(stem):
    return stem.split("_aug")[0]

# ========================
# 이미지 목록 확인
# ========================
images = sorted(SRC_IMG_DIR.glob("*.jpg"))

if not images:
    raise FileNotFoundError("dataset/images 폴더에 jpg 이미지가 없습니다.")

# 원본 ID별 그룹 생성
groups = {}

for img_path in images:
    origin_id = get_origin_id(img_path.stem)

    if origin_id not in groups:
        groups[origin_id] = []

    groups[origin_id].append(img_path)

origin_ids = list(groups.keys())
random.shuffle(origin_ids)
```

데이터 무결성 점검

지루하지만 반드시 해야 하는 일

목표 **영상–라벨 대응, 빈 파일, 클래스 ID, 좌표 범위를 확인한다**

```
import glob
from pathlib import Path

bad = set()

for p in G('dataset/labels/**/*.txt'):
    for ln in open(p):
        v = ln.split()
        if not v:
            continue

        if int(v[0]) not in (0, 1, 2):
            bad.add(p)

        if any(not 0 <= float(x) <= 1 for x in v[1:]):
            bad.add(p)

print('이상 파일:', len(bad))
```

확인 방법

- 라벨이 없는 영상은 의도된 배경 영상인지 라벨 누락인지 구분
- 영상이 없는 라벨 파일은 제거 또는 원본 영상 존재 여부 확인
- 빈 라벨 파일은 의도된 배경 라벨인지 확인
- 클래스 ID는 0, 1, 2만 허용
- polygon 좌표는 0~1 범위인지 확인
- 이상 파일이 발견되면 학습 전에 원인을 확인하고 수정

무결성 점검의 목적은 “오류 0개” 자체보다, 데이터셋이 의도한 구조대로 구성되었는지 확인하는 것이다.

분할과 data.yaml

이번 주에서 가장 중요한 단계

목표 **같은 원본에서 파생된 영상이 서로 다른 데이터셋에 포함되지 않도록 분할하고, data.yaml을 작성한다.**

```
import random
import shutil
from pathlib import Path
import yaml

# ========================
# 경로 설정
# ========================
SRC_IMG_DIR = Path("dataset/images")
SRC_LBL_DIR = Path("dataset/labels")
OUT_ROOT = Path("dataset_split")

# ========================
# 설정
# ========================
SEED = 42
SPLIT_RATIO = (0.8, 0.1, 0.1)


-> 다음 페이지
```

확인 방법

- 같은 원본에서 파생된 영상이 서로 다른 split에 포함되지 않았는지 확인
- 이미지와 라벨이 train / val / test 폴더에 올바르게 배치되었는지 확인
- 증강 데이터는 학습셋에만 포함
- data.yaml의 경로가 실제 폴더 구조와 일치하는지 확인
- 클래스 ID 순서가 0: crack, 1: efflorescence, 2: spall_detach와 일치하는지 확인

분할의 핵심은 개수 비율보다 데이터 누수를 막는 것이다. 같은 원본에서 파생된 영상은 반드시 동일한 split에 포함시킨다.

코드(2)

```
# =========================
# train / val / test 분할
# =========================
n = len(origin_ids)

n_train = int(n * SPLIT_RATIO[0])
n_val = int(n * SPLIT_RATIO[1])

train_ids = set(origin_ids[:n_train])
val_ids = set(origin_ids[n_train:n_train + n_val])
test_ids = set(origin_ids[n_train + n_val:])

split_ids = {
    "train": train_ids,
    "val": val_ids,
    "test": test_ids
}

print("원본 ID 수")
print("train:", len(train_ids))
print("val  :", len(val_ids))
print("test :", len(test_ids))

# =========================
# 출력 폴더 생성
# =========================
for split in ["train", "val", "test"]:
    (OUT_ROOT / "images" / split).mkdir(parents=True, exist_ok=True)
    (OUT_ROOT / "labels" / split).mkdir(parents=True, exist_ok=True)
```

```
# =========================
# 이미지 / 라벨 복사
# =========================
for split, ids in split_ids.items():

    for origin_id in ids:

        for img_path in groups[origin_id]:

            # 이미지 복사
            dst_img = OUT_ROOT / "images" / split / img_path.name
            shutil.copy2(img_path, dst_img)

            # 대응 라벨
            lbl_path = SRC_LBL_DIR / f"{img_path.stem}.txt"

            if lbl_path.exists():
                dst_lbl = OUT_ROOT / "labels" / split / lbl_path.name
                shutil.copy2(lbl_path, dst_lbl)
            else:
                print(f"[확인 필요] 라벨 없음: {img_path.name}")

# =========================
# data.yaml 생성
# =========================
data_yaml = {
    "path": str(OUT_ROOT.resolve()),
    "train": "images/train",
    "val": "images/val",
    "test": "images/test",
    "names": CLASS_NAMES
}

with open(OUT_ROOT / "data.yaml", "w", encoding="utf-8") as f:
    yaml.dump(
        data_yaml,
        f,
        allow_unicode=True,
        sort_keys=False
    )

print("\n분할 완료")
print("data.yaml:", OUT_ROOT / "data.yaml")
```

학습 실행

오늘의 마지막 단계

목표 **YOLOv8s-Seg 학습이 정상적으로 시작되고, 학습 손실이 출력되는지 확인한다**

```
from ultralytics import YOLO

# 사전학습된 YOLOv8s-Seg 모델 불러오기
model = YOLO("yolov8s-seg.pt")

# 모델 학습
model.train(
    data="data.yaml",       # 데이터셋 설정 파일
    epochs=60,              # 전체 학습 횟수
    imgsz=960,              # 입력 이미지 크기
    batch=4,                # GPU 메모리 부족 시 2로 조정
    optimizer="AdamW",      # 옵티마이저
    lr0=0.00015,            # 초기 학습률
    mosaic=0.0,             # Mosaic 증강 비활성화
    mixup=0.0,              # MixUp 증강 비활성화
    patience=20,            # Early Stopping 기준
    seed=42,                # 재현성을 위한 시드 고정
    project="runs",
    name="step3"
)
```

확인 방법

- 첫 epoch에서 box/seg/cls/dfl loss가 출력되는지 확인
- CUDA out of memory 발생 시 batch=2로 조정
- 수업 시간에는 약 3 epoch만 실행해 정상 동작을 확인
- 전체 학습 완료 후 best.pt를 다음 주 평가에 사용

분할의 핵심은 개수 비율보다 데이터 누수를 막는 것이다. 같은 원본에서 파생된 영상은 반드시 동일한 split에 포함시킨다.

학습 곡선 확인

학습이 제대로 되고 있는지

목표 **학습·검증 손실과 mask mAP의 변화를 함께 확인한다**

```
import pandas as pd
import matplotlib.pyplot as plt

# 학습 결과 불러오기
df = pd.read_csv("runs/step3/results.csv")
df.columns = [c.strip() for c in df.columns]

fig, ax = plt.subplots(1, 2, figsize=(12, 4))

# ① 학습 / 검증 Segmentation loss
for c in ["train/seg_loss", "val/seg_loss"]:
    if c in df.columns:
        ax[0].plot(df["epoch"], df[c], label=c)

ax[0].set_title("Segmentation Loss")
ax[0].set_xlabel("Epoch")
ax[0].set_ylabel("Loss")
ax[0].legend()
```

확인 방법

- 학습 손실이 꾸준히 줄면 정상
- 검증 손실이 흔들리는 것은 데이터가 적을 때 흔하다
- mAP가 전혀 오르지 않으면 라벨 포맷을 의심
- 이 그림을 리포트에 넣는다

학습 곡선 확인 코드(2)

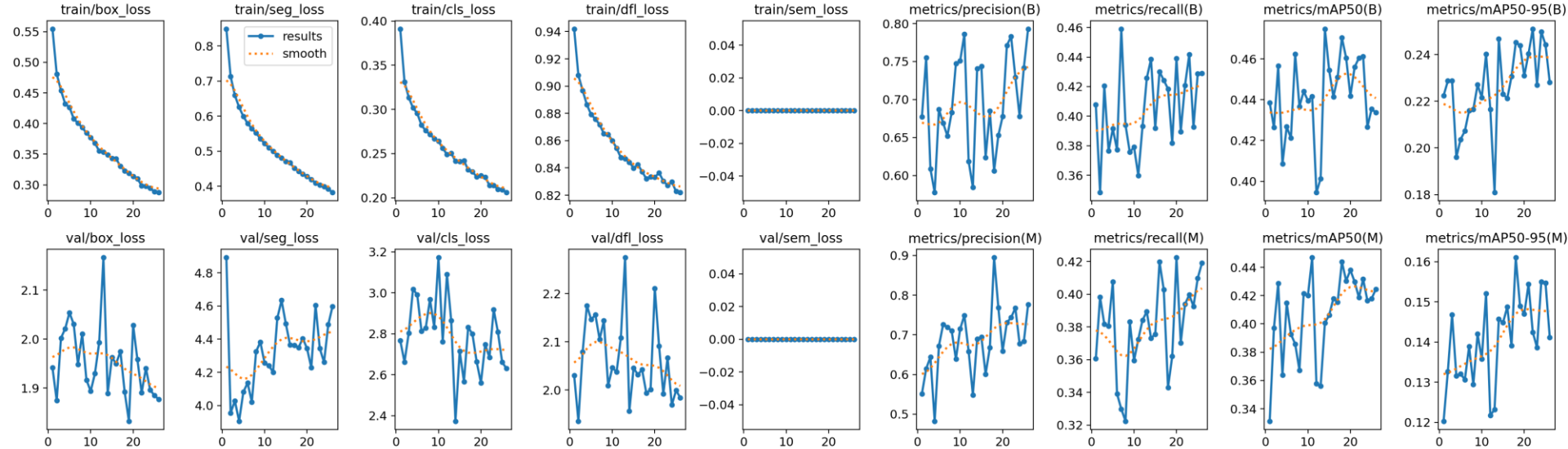
학습이 제대로 되고 있는지

```
# ② Mask mAP
for c in ["metrics/mAP50(M)", "metrics/mAP50-95(M)"]:
    if c in df.columns:
        ax[1].plot(df["epoch"], df[c], label=c)

ax[1].set_title("Mask mAP")
ax[1].set_xlabel("Epoch")
ax[1].set_ylabel("mAP")
ax[1].legend()

plt.tight_layout()
plt.savefig("curve.png", dpi=150)
plt.show()
```

연구 결과 예시


train/box_loss
train/seg_loss
results
smooth
train/cls_loss
train/dfl_loss
train/sem_loss
metrics/precision(B)
metrics/recall(B)
metrics/mAP50(B)
metrics/mAP50-95(B)
val/box_loss
val/seg_loss
val/cls_loss
val/dfl_loss
val/sem_loss
metrics/precision(M)
metrics/recall(M)
metrics/mAP50(M)
metrics/mAP50-95(M)

여기까지 됐으면 정상

스스로 확인하는 체크포인트

오늘 안에 확인할 것

- 제공된 실습 데이터 중 일부 샘플의 폴리곤 주석을 완료했다
- 라벨 파일이 세그멘테이션 포맷임을 확인했다
- 데이터 무결성 점검 결과를 확인하고, 이상 파일이 있으면 원인을 수정했다
- 같은 원본에서 파생된 영상이 서로 다른 split에 포함되지 않도록 분할했다
- data.yaml 을 만들고 클래스 순서를 확인했다
- YOLOv8s-Seg 학습이 정상적으로 시작되고 주요 loss가 출력된다
- curve.png에서 segmentation loss와 mask mAP의 변화를 확인했다

위 항목을 모두 확인했다면 이번 주 목표는 달성이다. 이번 주는 학습 파이프라인의 정상 동작을 확인하고, 최종 성능 평가는 다음 주에 진행한다.

자주 나는 오류와 해결

대부분 여기서 막힌다

증상	원인	해결
라벨 한 줄이 5개 값으로 구성됨	Detection 포맷으로 export했을 가능성	Segmentation 포맷으로 다시 export하고 polygon 좌표가 포함됐는지 확인
mAP가 0에서 거의 오르지 않음	클래스 ID 불일치, 좌표 범위 오류, 라벨 포맷 문제	무결성 점검을 다시 실행하고 라벨 파일 예시를 직접 확인
CUDA out of memory	GPU 메모리 부족	우선 batch=2로 줄이고, 그래도 부족하면 imgsz 조정
영상을 못 찾음	data.yaml 경로 또는 폴더 구조 불일치	실제 데이터셋 경로와 images/train, images/val, images/test 구조 확인
학습이 매우 느림	GPU가 사용되지 않고 CPU로 실행	torch.cuda.is_available()과 device 설정 확인
성능이 비현실적으로 높음	같은 원본에서 파생된 영상이 train/val에 동시에 포함됨	원본 ID 기준 분할이 제대로 되었는지 다시 확인

포맷 오류와 데이터 분할 오류는 실습에서 자주 발생하므로, 학습 전에 반드시 확인한다.

이번 주 제출물

다음 주 강의 전까지

① 주석 결과	직접 주석한 영상 수 · 클래스별 인스턴스 수
② 포맷 확인	라벨 파일 첫 줄 출력 화면
③ 무결성 점검	무결성 점검 결과 및 이상 항목 수정 여부
④ 분할 코드	원본 ID 기준으로 나눈 부분을 표시
⑤ 학습 곡선	curve.png 또는 results.png의 segmentation loss와 mask mAP 변화
⑥ 설정 근거	imgsz 960 · Mosaic/MixUp 비활성화 이유를 간단히 서술
제출 형식	PDF 또는 노트북 1개 · best.pt · 실습 코드/결과 파일

④번을 원본 단위 분할 코드를 반드시 표시할 것. 같은 원본에서 파생된 영상이 서로 다른 split에 포함되지 않았음을 확인한다.

다음 주에 할 일

3주차 미리보기

다음 주 강의 내용

- mask mAP50과 mAP50-95
- 이미지 단위 ON/OFF 평가
- 검증셋 기반 클래스별 신뢰도 임계값 탐색
- FP·FN 오류 사례 분석

다음 주 실습에서 만들 것

- 검증셋 임계값 F1 곡선
- 클래스별 최적 임계값
- 검증셋에서 선택한 임계값을 테스트셋에 고정 적용
- FP·FN 사례 분석

미리 해두면 좋은 것

- 전체 학습을 완료해 best.pt를 확보해 둘 것 — 다음 주 평가는 해당 가중치를 사용한다
- best.pt 를 반드시 저장해 둘 것
- 학습에 걸린 시간을 기록해 둘 것